Lab 3. Creating and Managing Schema Objects

Content

- Data definition language
- Oracle datatypes
- Creating and managing tables
- Creating and managing indexes
- Integrity constraints
- Creating and managing views
- Exercises

Data definition language

Statements that create, modify, or change schema objects are called Data Definition Language (DDL) statements. The CREATE, ALTER, and DROP statements are examples of DDL statements. We create a new schema object using the CREATE statement, modify existing schema objects using the ALTER statement, and remove the schema object from the schema using the DROP statement. We will explore more on DDL statements in the following topics.

Oracle datatypes

Table 1. Built-in Data Type Summary

No	Datatype	Description
1	CHAR [(*size* [BYTE | CHAR])]	**Fixed-length** character data of length *size* bytes or characters. Maximum *size* is 2000 bytes or characters. Default and minimum *size* is 1 byte.
2	NCHAR[(*size*)]	**Fixed-length Unicode** character data of length *size* characters. The number of bytes can be up to two times *size* for AL16UTF16 encoding and three times *size* for UTF8 encoding. Maximum *size* is determined by the national character set definition, with an upper limit of 2000 bytes. Default and minimum *size* is 1 character.
3	VARCHAR2(size [BYTE | CHAR])	**Variable-length** character string having maximum length *size* bytes or characters. Maximum *size* is 4000 bytes or characters, and minimum is 1 byte or 1 character. You must specify *size* for VARCHAR2. BYTE indicates that the column will have byte length semantics. CHAR indicates that the column will have character semantics.
4	NVARCHAR2(size)	**Variable-length Unicode** character string having maximum length *size* characters. The number of bytes can be up to two

		times *size* for AL16UTF16 encoding and three times *size* for UTF8 encoding. Maximum *size* is determined by the national character set definition, with an upper limit of 4000 bytes. You must specify *size* for NVARCHAR2.
5	CLOB	A **character large object** containing single-byte or multibyte characters. Both fixed-width and variable-width character sets are supported, both using the database character set. Maximum size is (4 gigabytes - 1) * (database block size).
6	NCLOB	A **character large object** containing **Unicode characters**. Both fixed-width and variable-width character sets are supported, both using the database national character set. Maximum size is (4 gigabytes - 1) * (database block size).
7	BLOB	A **binary** large object. Maximum size is (4 gigabytes - 1) * (database block size).
8	BFILE	Binary data stored in an **external file** (up to 4 GB).
9	NUMBER [(p [, s])]	Number having precision *p* and scale *s*. The precision *p* can range from 1 to 38. The scale *s* can range from -84 to 127. Both precision and scale are in decimal digits. A NUMBER value requires from 1 to 22 bytes.
10	FLOAT [(p)]	A subtype of the NUMBER data type having precision *p*. A FLOAT value is represented internally as NUMBER. The precision *p* can range from 1 to 126 binary digits. A FLOAT value requires from 1 to 22 bytes.
11	DATE	Valid date range from January 1, 4712 BC, to December 31, 9999 AD. The default format is determined explicitly by the NLS_DATE_FORMAT parameter or implicitly by the NLS_TERRITORY parameter. The size is fixed at 7 bytes. This data type contains the datetime fields YEAR, MONTH, DAY, HOUR, MINUTE, and SECOND. It does not have fractional seconds or a time zone.
12	TIMESTAMP [(precision)]	Year, month, and day values of date, as well as hour, minute, and second values of time, where *precision* is the number of digits in the fractional part of the SECOND datetime field. Accepted values of *precision* are 0 to 9. The default is 6. The default format is determined explicitly by the NLS_TIMESTAMP_FORMAT parameter or implicitly by the NLS_TERRITORY parameter. The size is 7 or 11 bytes, depending on the precision. This data type contains the datetime fields YEAR, MONTH, DAY, HOUR,

		MINUTE, and SECOND. It contains fractional seconds but does not have a time zone.
13	TIMESTAMP [(precision)] WITH LOCAL TIME ZONE	All values of TIMESTAMP WITH TIME ZONE, with the following exceptions: ● Data is normalized to the database time zone when it is stored in the database. ● When the data is retrieved, users see the data in the session time zone. The default format is determined explicitly by the NLS_TIMESTAMP_FORMAT parameter or implicitly by the NLS_TERRITORY parameter. The size is 7 or 11 bytes, depending on the precision.
14	RAW(size)	Raw **binary data** of length *size* bytes. Maximum *size* is 2000 bytes. You must specify *size* for a RAW value.
15	LONG RAW	Raw **binary data** of variable length up to 2 gigabytes.
16	UROWID [(size)]	Base 64 string representing the logical address of a row of an index-organized table. The optional size is the size of a column of type UROWID. The maximum size and default is 4000 bytes.

Creating and managing tables

Creating tables

We use the CREATE TABLE statement to create a new table in the Oracle Database. The following illustrates the basic syntax of the CREATE TABLE statement:

```
CREATE TABLE schema_name.table_name (
    column_1 data_type column_constraint,
    column_2 data_type column_constraint,
    ...
    table_constraint
 );
```

In this syntax:

- First, specify the *table name* and *schema name* to which the new table belongs on the CREATE TABLE clause.
- Second, list all columns of the table within the parentheses. If a table has multiple columns, you must separate them by commas (,). A column definition includes the column name followed by its data type e.g., NUMBER, VARCHAR2, and a column constraint such as NOT NULL, primary key, check.

- Third, add table constraints if applicable, e.g., primary key, foreign key, and check.

In Oracle, the total number of columns cannot be more than 32.

The following example shows how to create a new table named persons in the **ot** schema:

```
CREATE TABLE ot.persons(
    person_id NUMBER NOT NULL,
    first_name VARCHAR2(50) NOT NULL,
    last_name VARCHAR2(50) NOT NULL,
    PRIMARY KEY (person_id)
);
```

Viewing tables' structure

To view a table structure, we use the DESC command as follows:

```
DESC table_name;
```

For example:

```
DESC persons;

Name        Null     Type
----------- -------- ------------
PERSON_ID   NOT NULL NUMBER
FIRST_NAME  NOT NULL VARCHAR2(50)
LAST_NAME   NOT NULL VARCHAR2(50)
```

Altering tables

To modify the structure of an existing table, you use the ALTER TABLE statement. The following illustrates the syntax:

```
ALTER TABLE table_name action;
```

The ALTER TABLE statement allows you to:

- Add one or more columns:

  ```
  ALTER TABLE table_name
        ADD column_name type constraint;
  ```

  Example:

  ```
  ALTER TABLE persons
        ADD birthdate DATE NOT NULL;
  ```

  To add multiple columns to a table at the same time, you place the new columns inside the parenthesis as follows:

```
ALTER TABLE table_name
ADD (
    column_name type constraint,
    column_name type constraint,
    ...
);
```

- Modify column definition:

```
ALTER TABLE table_name
   MODIFY column_name type constraint;
```

Example:

```
ALTER TABLE persons MODIFY birthdate DATE NULL;
```

To modify multiple columns, you use the following syntax:

```
ALTER TABLE table_name
  MODIFY (column_1 type constraint,
          column_1 type constraint,
         ...
);
```

- Drop one or more columns:

```
ALTER TABLE table_name
      DROP COLUMN column_name;
```

To drop multiple columns at the same time, you use the syntax below:

```
ALTER TABLE table_name
      DROP (column_1,column_2,...);
```

- Rename columns

```
ALTER TABLE table_name
      RENAME COLUMN column_name TO new_name;
```

- Rename table

```
ALTER TABLE table_name
      RENAME TO new_table_name;
```

Creating and managing indexes

Indexes are created in a database to locate relevant information quickly. When properly used, indexes will speed up SQL execution while reducing disk I/O and memory access. To better understand what an index is, think of the index of words at the back of any book. If you want to locate information quickly, you should refer to the index of words and navigate to that page. Indexes in Oracle are schema objects that are stored separately.

To create a new index for a table, you use the `CREATE INDEX` statement as follows:

```
CREATE INDEX index_name
      ON table_name(column1[, column2,...]);
```

In this syntax:

- First, specify the name of the index. The index name should be meaningful and include table alias and column name(s) where possible, along with the suffix `_I` such as: `<table_name>_<column_name>_I`
- Second, specify the table's name followed by one or more indexed columns surrounded by parentheses.

When you create a new table with a **primary key**, Oracle automatically creates a new index for the primary key columns.

The following statement creates an index for the `last_name` column of the `members` table:

```
CREATE INDEX members_last_name_i
      ON members(last_name);
```

To view all indexes of a table, we query from the `all_indexes` view:

```
SELECT index_name, index_type, visibility, status
      FROM all_indexes
      WHERE table_name = 'MEMBERS';
```

To remove an index, we use the `DROP INDEX` statement:

```
DROP INDEX index_name;
```

For example, to drop the `members_last_name_i` index, we use the following statement:

```
DROP INDEX members_last_name_i;
```

Integrity constraints

Data stored in the database must adhere to certain business rules. An integrity constraint defines a business rule for a table column. When the integrity constraint is enabled, Oracle will enforce the business rule. Integrity constraints are stored as part of the table definition within the database.

The following are the integrity constraints:

- NOT NULL: The NOT NULL constraint enforces a column not to accept null values.
- CHECK: The CHECK constraint is used to limit the values that are accepted by a column.
- UNIQUE: The UNIQUE constraint ensures that no duplicate values are entered in the column.
- PRIMARY KEY: The PRIMARY KEY constraint uniquely identifies a row in a table. PRIMARY KEY can be thought of as a combination of the UNIQUE key and NOT NULL constraints. Furthermore, a table can have only one primary key.
- FOREIGN KEY: A foreign key column is used to establish a link between two tables. The FOREIGN KEY constraint on a column ensures that the value in that column is found in the primary key of another table.

PRIMARY KEY

Typically, you create a primary key for a table when you create that table. In addition, you can add a primary key to a table after the fact by using the ALTER TABLE statement.

The following `CREATE TABLE` statement creates the `purchase_orders` table:

```
CREATE TABLE purchase_orders (
    po_nr NUMBER PRIMARY KEY,
    vendor_id NUMBER NOT NULL,
    po_status NUMBER(1,0) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL
);
```

The following statement creates a primary key that consists of **multiple columns**:

```
CREATE TABLE purchase_order_items (
    po_nr NUMBER NOT NULL,
    item_nr NUMBER NOT NULL,
    product_id NUMBER NOT NULL,
    quantity NUMBER NOT NULL,
    purchase_unit NUMBER NOT NULL,
    buy_price NUMBER (9,2) NOT NULL,
    delivery_date DATE,
    PRIMARY KEY (po_nr, item_nr)
);
```

To add a primary key constraint to an existing table, we use the ALTER TABLE statement as follows:

```
ALTER TABLE table_name
      ADD CONSTRAINT constraint_name
      PRIMARY KEY (column1, column2, ...);
```

To drop a PRIMARY KEY constraint from a table, we use the following ALTER TABLE statement:

```
ALTER TABLE table_name
      DROP CONSTRAINT primary_key_constraint_name;
```

Or:

```
ALTER TABLE table_name
      DROP PRIMARY KEY;
```

FOREIGN KEY

The syntax of creating a foreign key constraint when creating a table:

```
CREATE TABLE child_table (
    ...
    [ CONSTRAINT fk_name ]
    FOREIGN KEY(col1, col2,...) REFERENCES parent_table(col1,col2)
    ON DELETE [ CASCADE | SET NULL ]
);
```

In this syntax:

- The CONSTRAINT clause is optional. If you omit it, Oracle will assign a system-generated name to the foreign key constraint.
- ON DELETE CASCADE: if a row in the parent is deleted, then all the rows in the child table that reference the removed row will be deleted.
- ON DELETE SET NULL: if a row in the parent is deleted, then all the rows in the child table reference the removed row will be set to NULL for the foreign key columns.

To add a foreign key constraint to an existing table, we use the ALTER TABLE statement as follows:

```
ALTER TABLE child_table
      ADD CONSTRAINT fk_name
      FOREIGN KEY (col1,col2) REFERENCES parent_table(col1,col2);
```

To drop a foreign key constraint, we use the ALTER TABLE statement below:

```
ALTER TABLE child_table
      DROP CONSTRAINT fk_name;
```

CHECK

Typically, you create a check constraint on a column when you create the table:

```
CREATE TABLE table_name (
    ...
    column_name data_type
        [ CONSTRAINT check_constraint_name] CHECK (expression),
    ...
);
```

We can use the out-of-line constraint syntax as follows:

```
CREATE TABLE table_name (
    ...,
    CONSTRAINT check_constraint_name CHECK (expresssion)
);
```

The following example creates the parts table whose **buy prices are positive**:

```
CREATE TABLE parts (
    part_id NUMBER,
    part_name VARCHAR2(255) NOT NULL,
    buy_price NUMBER(9,2) CHECK(buy_price > 0),
    PRIMARY KEY(part_id)
);
```

We can give a check constraint an **explicit name**:

```
CREATE TABLE parts (
    part_id NUMBER GENERATED BY DEFAULT AS IDENTITY,
    part_name VARCHAR2(255) NOT NULL,
    buy_price NUMBER(9,2)
       CONSTRAINT check_positive_buy_price CHECK(buy_price > 0),
    PRIMARY KEY(part_id)
);
```

To add a check constraint to an existing table, we use the `ALTER TABLE ADD CONSTRAINT` statement as follows:

```
ALTER TABLE table_name
     ADD CONSTRAINT check_constraint_name CHECK(expression);
```

To drop a check constraint, we use the `ALTER TABLE DROP CONSTRAINT` statement as follows:

```
ALTER TABLE table_name
      DROP CONSTRAINT check_constraint_name;
```

Creating and managing views

A view is a stored query. You write and save a SQL query in the database as a view. A view can reference a single table/view, or multiple tables/views.

To create a new view in a database, you use the following Oracle `CREATE VIEW` statement:

```
CREATE [OR REPLACE] VIEW view_name [(column_aliases)] AS
      defining-query
      [WITH READ ONLY]
      [WITH CHECK OPTION]
```

In this syntax:

- `OR REPLACE:` to replace the definition of the existing view.
- `Column-aliases`: to name the column of the defining query, usually used when the column names of the defining query contain functions or expressions.
- `defining-query`: a `SELECT` statement that defines the columns and rows of the view.
- `WITH READ ONLY`: to prevent the underlying tables from changing through the view.
- `WITH CHECK OPTION`: to protect the view from any changes to the underlying table that would produce rows that are not included in the defining query.

The following statement creates a view named **`employee_yos`** based on the **`employees`** table. The view shows the `employee id`, `name`, and `years of service`:

```
CREATE VIEW employee_yos AS
SELECT
    employee_id,
    first_name || ' ' || last_name full_name,
    FLOOR( months_between( CURRENT_DATE, hire_date )/ 12 ) yos
FROM
    employees;
```

The following query returns employees whose years of service are 15:

```
SELECT *
      FROM employee_yos
      WHERE yos = 15
      ORDER BY full_name;
```

The following statement creates a view named backlogs whose defining query includes join clauses that join three tables: orders, order_items, and products.

```
CREATE OR REPLACE VIEW backlogs AS
      SELECT product_name,EXTRACT(YEAR FROM order_date) YEAR,
          SUM( quantity * unit_price ) amount
      FROM
          orders
      INNER JOIN order_items
              USING(order_id)
      INNER JOIN products
              USING(product_id)
      WHERE status = 'Pending'
      GROUP BY EXTRACT(YEAR FROM order_date), product_name;
```

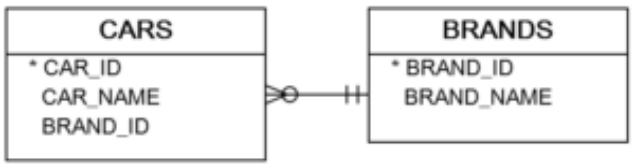
To remove a view from the database, you use the following DROP VIEW statement:

```
DROP VIEW schema_name.view_name
      [CASCADE CONSTRAINT];
```

The CASCADE CONSTRAINT clause will drop all referential integrity constraints referring to the view's primary and unique keys. If we don't do so, then the DROP VIEW statement will fail if such constraints exist. If other views reference a view, Oracle will mark these objects as INVALID but will not remove them.

A view behaves like a table because you can query data from it. However, we cannot always manipulate data via views. A view is **updatable** if the statement against the view can be translated into the corresponding statement against the underlying table.

For example, given the following tables:



The following statement creates an **updatable view** named cars_master:

```
CREATE VIEW cars_master AS
      SELECT car_id, car_name
      FROM cars;
```

It's possible to delete a row from the cars table or update any column values via the cars_master view, for example:

```
DELETE
```

```
      FROM cars_master
      WHERE car_id = 1;


UPDATE cars_master
      SET car_name = 'Audi RS7 Sportback'
      WHERE car_id = 2;
```

However, inserting a new row into the cars table via the `cars_master` view is not possible because the cars table has a null column (`brand_id`) without a default value.

```
INSERT INTO cars_master
      VALUES('Audi S1 Sportback');
```

Oracle issued an error:

SQL Error: ORA-00947: not enough values

Here are some examples of updatable join view restrictions:

- SQL statements, e.g., INSERT, UPDATE, and DELETE, are only allowed to modify data from a single base table.
- For an `INSERT` statement, all columns listed in the `INTO` clause must belong to a key-preserved table.
- For an `UPDATE` statement, all columns in the `SET` clause must belong to a key-preserved table.
- For a `DELETE` statement, if the join results in more than one key-preserved table, the Oracle deletes from the first table in the `FROM` clause.

Besides these restrictions, Oracle also requires that the defining query does not contain any of the following elements:

- Aggregate functions e.g., `AVG`, `COUNT`, `MAX`, `MIN`, and `SUM`.
- `DISTINCT` operator.
- `GROUP BY` clause.
- `HAVING` clause.
- Set operators e.g., `UNION`, `UNION ALL`, `INTERSECT`, and `MINUS`.
- `START WITH` or `CONNECT BY` clause
- `ROWNUM` pseudo-column

Exercises

In this exercise, we will design a database for an **Employee Management Application** that stores information about **employees**, **departments**, and **employee leaves** in various database tables. It provides the structure and sample data for tables to store department,

employee, leave type, and employee leave information. It also provides examples of SQL queries that can be run on the tables to retrieve and analyze the employee data. Following are the requirements for designing the database for this application.

1. Create the following tables:

No	Table name	Description
1.	DEPT	Stores the details of departments of the company
2.	EMPLOYEE	Stores information about all the employees of the company
3.	LEAVES	Stores the information about types of leaves available
4.	EMP_LEAVE	Stores information about leaves taken by the employees

The structure of the above tables is described as follows:

DEPT table:

No	**Column name**	**Datatype**	**Description**	**Constraint**
1.	DEPT_NO	Number(2)	Department number	Primary key
2.	DEPT_NAME	Varchar2(20)	Department name	Unique
3.	HOD	Varchar2(20)	Head of the department	

EMPLOYEE table:

No	**Column name**	**Datatype**	**Description**	**Constraint**
1.	EMP_NO	Number(5)	Employee number	Primary key
2.	EMP_NAME	Varchar2(20)	Employee name	
3.	SAL	Number(6)	Basic salary	>= 1.000
4.	DEPT_NO	Number(2)	Department to which the employee belongs	Ref to the DEPT(DEPT_NO)
5.	DJ	Date	Date of joining the company	
6.	DEG	Varchar2(20)	Designation of the employee	

LEAVE table:

No	**Column name**	**Datatype**	**Description**	**Constraint**
1.	LEAVE_TYPE	Char(1)	Code for the type of leave	Primary key

2.	LEAVE_NAME	Varchar2(20)	Description of the type of leave	
3.	NO_LEAVES	Number(2)	Number of leaves allotted to each employee for a leave type	<= 20

EMP_LEAVE table:

No	Column name	Datatype	Description	Constraint
1.	EMP_NO	Number(5)	Employee number of the employee who has taken leave	Primary key FK, ref to EMPLOYEE(EMP_NO)
2.	LEAVE_TYPE	Char(1)	Type of the leave taken by the employee	NOT NULL FK, ref to LEAVES(LEAVE_TYPE)
3.	START_DATE	Date	Starting date of the leave	Primary key
4.	END_DATE	Date	Ending date of the leave	>= START_DATE

2. Add data to the above tables:
   DEPT table:

DEPT_NO	DEPT_NAME	HOD
1	MAINFRAME	GEORGE
2	CLIENT/SERVER	BILL
3	SYSTEMS	GARRY
4	INTERNET	PAUL
5	ACCOUNTS	ANDY

EMPLOYEE table:

EMP_NO	EMP_NAME	SAL	DEPT_NO	DJ	DESG
101	GEORGE	12000	1	12-JUL-2001	PM
102	BILL	12000	2	14-JUL-2001	PM
103	GARRY	15000	3	1-JUL-2001	PM
104	PAUL	11000	4	2-JUL-2001	PL
105	ANDY	7000	5	25-JUN-2001	AM
106	KEATS	10000	1	17-JUL-2001	SA
107	JOEL	8000	2	15-JUL-2001	SP
108	ROBERTS	7500	2	15-JUL-2001	PRO

109	HERBERT	8000	4	22-JUL-2001	SA
110	MICHEAL	6000	4	15-JUL-2001	PRO

LEAVE table:

LEAVE_TYPE	LEAVE_NAME	NO_LEAVES
C	CASUAL	15
E	EARNING	5
O	OVERTIME	5
S	SICK	15

EMP_LEAVES table:

EMP_NO	LEAVE_TYPE	START_DATE	END_DATE
102	S	23-JUL-2001	25-JUL-2001
104	C	24-JUL-2001	25-JUL-2001
104	S	28-JUL-2001	29-JUL-2001
101	C	27-JUL-2001	28-JUL-2001
106	O	28-JUL-2001	29-JUL-2001
109	C	1-AUG-2001	2-AUG-2001
103	C	2-AUG-2001	5-AUG-2001
105	S	17-AUG-2001	NULL
108	S	23-AUG-2001	NULL

3. Write queries related to Employees Management Application:
    a. Display employees who have joined in the last 15 days.
    b. Display the details of employees who have taken more than 10 sick leave days or more than 15 leave days.
    c. Change the `deptno` of employee 104 to `deptno` of the 'Internet' dept.
    d. Delete details of leaves taken by employees whose `empno` is the highest `empno`.
    e. Change the `deptno` of employee 104 to `deptno` of the 'Internet' dept.
4. Write an SQL statement to create a view named vw_ManyLeavesEmp that displays the `empno` of the employee who has taken more than 2 leaves in the current month.
5. Write an SQL statement to create a view named vw_LeavesPriodMonth that displays details of leaves where the leave started in the previous month and is not yet completed.
6. Explore and write a report about the index in Oracle (in Powerpoint or Word format):
    a. What is an index?
    b. What types of index does Oracle support?
    c. Demonstrate the effect of index in querying data in Oracle (Hint: using the execution plan).

d. References:

- SQL Turning Guide: https://docs.oracle.com/en/database/oracle/oracle-database/18/tgsql/sql-tuning-guide.pdf
- Database Tuning: Principles, Experiments, and Troubleshooting Techniques: https://epdf.pub/database-tuning-principles-experiments-and-troubleshooting-techniquese40ea453066be9eb45366ff610cdce4449233.html
- Indexes and Index-Organized Tables: https://docs.oracle.com/en/database/oracle/oracle-database/12.2/cncpt/indexes-and-index-organized-tables.html
- SQL Indexing and Tuning: https://use-the-index-luke.com/
- Oracle Indexes Reduce The Performance: Explainer: https://bytescout.com/blog/oracle-indexes-reduce-the-performance-explainer.html
- Indexing LIKE Filters: https://use-the-index-luke.com/sql/where-clause/searching-for-ranges/like-performance-tuning